ПЛАН АВТОМАТИЗАЦИИ ТЕСТИРОВАНИЯ

American Eagle Java Automation

Объект: https://www.ae.com/

Параметр	Значение
Документ	Test Plan / Automation Test Plan
Проект	American_Eagle_java_automation
Ответственный	Илья Сидорычев
Версия документа	3.1
Дата актуализации	12.10.2026
Статус	Актуальный после overlay и Gradle refactoring
Тестовый набор	55 test cases: 20 API + 35 UI

1. Цели автоматизации

- Ускорить проверку критически важных пользовательских и API-сценариев.
- Снизить объём повторяющихся ручных проверок и влияние человеческого фактора.
- Раннее выявлять регрессионные дефекты после изменений в тестируемом продукте и тестовой инфраструктуре.
- Обеспечить воспроизводимый запуск API- и UI-наборов локально и в CI.
- Формировать единый Allure Report с историей запусков и публиковать его через GitHub Pages.
- Разделять дефекты продукта, технические ограничения anti-bot и инфраструктурные падения автотестов.

2. Объект и контекст тестирования

American Eagle (ae.com) - реальный production e-commerce сайт. Проект охватывает Web UI и REST API, связанные с навигацией, поиском, товарами, пользовательскими состояниями и корзиной.

- Тестирование выполняется против действующего production-окружения.
- Используются только безопасные сценарии, не приводящие к реальной покупке или оплате.
- Поведение сайта может изменяться независимо от тестового проекта, поэтому тесты адаптируются к изменениям production-системы.
- Anti-bot protection может ограничивать отдельные автоматизированные пользовательские сценарии.
- Сетевые и production-сбои рассматриваются отдельно от функциональных дефектов продукта.

3. Scope - что входит в автоматизацию

3.1 UI

Область	Что проверяется	Категории
Home / Header / Footer	Главная страница, browser title, header, footer, основные навигационные элементы.	UI, smoke / extended
Men's catalog	Каталог, карточки товара, выбор доступного товара/размера, корзина, quantity, subtotal, free shipping, удаление.	UI, extended
Search	Открытие поиска, существующий товар, known defect для бессмысленного запроса.	UI, smoke / negative
Registration	Форма регистрации, email, обязательные поля, Birthday, Terms and Conditions. Успешная отправка ограничена anti-bot.	UI, extended / positive / negative
Sign In	Позитивные и негативные проверки формы входа; часть серверных сценариев ограничена anti-bot.	UI, extended / positive / negative

3.2 REST API

Область	Что проверяется	Категории
Token API	Guest token, access_token, token_type, scope, expires_in; запрос без Authorization -> 401.	API, positive / negative
Browse / Product API	Категории, динамические productId/SKU, invalid category; запрос без Bearer token -> 401.	API, positive / negative
Inventory API	Inventory по реальному productId, SKU, invalid product ID; запрос без Bearer token -> 401.	API, positive / negative
Bag API	Получение корзины, add/get/update/delete, несколько товаров, invalid SKU и другие negative scenarios.	API, positive / negative
E2E API	Полный lifecycle: productId -> SKU -> add -> verify -> update -> wait -> delete -> verify.	API, E2E

3.3 Размер тестового набора

Набор	Test cases	Статус / примечание
API	20	Полный API suite успешно пройден после финального рефакторинга.
UI	35	Включает 7 @Disabled anti-bot сценариев и 1 активный @KnownDefect сценарий.
Всего	55	Полный JUnit-набор проекта.

4. Out of Scope - что не тестируется

- Оформление реального заказа и любые операции оплаты.
- Проверка внутренних баз данных и прямой доступ к backend-хранилищам.
- Нагрузочное и стресс-тестирование production-сайта.
- Security / penetration testing.
- Деструктивные сценарии, способные повлиять на реальных пользователей или production-данные.
- Стабильное подтверждение успешной регистрации/авторизации на сервере, если запрос блокируется anti-bot protection.

5. Типы, уровни и теги тестов

Тег / механизм	Назначение
UI	Web UI тесты через Selenium WebDriver
API	REST API тесты через REST Assured
smoke	Короткий набор ключевых проверок
extended	Расширенные функциональные сценарии
positive	Позитивные сценарии
negative	Негативные сценарии
E2E	Сквозной API lifecycle-сценарий
@KnownDefect	Активный сценарий подтверждённого дефекта; assertion failure -> aborted/skipped, unexpected pass -> failure
@Disabled	Технически недоступный сценарий, например из-за anti-bot protection

6. Тестовые данные и управление состоянием

- Product IDs и SKU по возможности подбираются динамически через Browse / Inventory API.
- Guest token создаётся/получается программно и используется для API-сценариев.
- Перед API-тестами состояние корзины нормализуется, чтобы тесты были независимыми друг от друга.

- Credentials не хранятся в репозитории; локально guest Basic credential хранится во внешнем пользовательском Gradle-файле ~/.gradle/gradle.properties и автоматически передаётся тестовой JVM как guest.header.auth; в CI используется GitHub Secrets.
- При eventual consistency используется Awaitility с ограниченным временем ожидания и polling.

7. Технологический стек

Технология	Назначение
Java 21	Основной язык
JUnit 5	Тестовый фреймворк и Extensions
Selenium WebDriver 4.50.0	UI-автоматизация
REST Assured	REST API тестирование
AssertJ	Fluent assertions / SoftAssertions
Awaitility 4.3.0	Ожидание eventual consistency
Gradle 8.10	Сборка, зависимости и test tasks

7. Технологический стек - продолжение

Технология	Назначение
Owner	Конфигурация окружений
Lombok	DTO и сокращение boilerplate-кода
Allure Report	Отчётность
Git / GitHub	Контроль версий
GitHub Actions	CI
GitHub Pages	Публикация Allure Report
Selenium Standalone Chrome	Удалённый запуск UI в CI
IntelliJ IDEA	Среда разработки

8. Архитектура автоматизации

- Page Object Model и Component-классы для UI.
- Reusable Steps layer для пользовательских последовательностей.
- Controller-слой для API-вызовов.
- DTO-модели для сериализации/десериализации API-данных.
- JUnit 5 Extensions для подготовки контекста, guest token и Known Defect обработки.
- Steps / Support / Utils для переиспользуемой логики.
- Owner-конфигурация для работы с окружениями.
- Allure annotations для шагов и severity.
- Gradle tasks и JUnit tags для запуска отдельных наборов; общая конфигурация всех Test tasks централизована через tasks.withType(Test).configureEach.

8.1 KnownDefectExtension

KnownDefectExtension реализует strict-поведение для подтверждённого дефекта поиска:

- Ожидаемая AssertionError означает воспроизведение дефекта и переводит тест в aborted / skipped.
- Если тест неожиданно проходит, он завершается failure с требованием проверить, исправлен ли дефект.
- TimeoutException, WebDriverException, locator problems и другие технические ошибки не преобразуются в Known Defect. UI overlay-логика централизована в BasePage: сначала закрывается location modal, затем cookie banner и Bloomreach email popup. Для custom Test tasks явно задаются testClassesDirs и classpath, что устраняет Gradle 9 deprecation warnings.

9. Окружение

Среда	Описание
Локально	Windows 10 x64; Java 21; актуальная версия Google Chrome; IntelliJ IDEA; Gradle Wrapper.
CI	GitHub-hosted Ubuntu runner; Java 21; Selenium Standalone Chrome; Gradle Wrapper.
SUT	Публичный production-сайт American Eagle и публично доступные API, используемые проектом.

10. Стратегия запуска

Основные Gradle tasks:

```
.\gradlew apiTests --rerun-tasks
.\gradlew uiTests --rerun-tasks
.\gradlew allTests
.\gradlew smoke
.\gradlew positive
.\gradlew negative
.\gradlew e2e
.\gradlew homePageTestsParallel
```

Глобальный parallel run всего UI-набора намеренно не используется: на реальном production-сайте и локальном Chrome параллельное выполнение показало меньшую стабильность. `homePageTestsParallel` оставлен как отдельный ограниченный экспериментальный task.

11. CI/CD стратегия

```
api-tests -> api-allure-results \
                                -> allure-report -> GitHub Pages
ui-tests  -> ui-allure-results /
```

- API и UI выполняются в отдельных jobs.
- API job получает AE_GUEST_AUTH из GitHub Secrets и передаёт credential тестовой JVM; локально эквивалентное значение читается из пользовательского Gradle configuration.
- UI job использует Selenium Standalone Chrome и remote WebDriver.
- Каждый job сохраняет собственный build/allure-results как artifact.
- Отдельный allure-report job объединяет результаты и восстанавливает историю.
- Готовый Allure Report публикуется через GitHub Pages.

12. Allure Reporting

- Единый отчёт объединяет результаты API и UI.
- В отчёте доступны suites, test cases, severity, tags, шаги, ошибки, stack traces, attachments и история запусков.
- Публикуемая версия отчёта обновляется после успешного report job.
- Текущая ссылка: https://ismeneger.github.io/American_Eagle_java_automation/

13. Anti-bot, ограничения и Known Defect стратегия

- Production-сайт использует anti-bot protection, которая может блокировать автоматизированную регистрацию и авторизацию.
- В текущем UI-наборе 7 test cases помечены @Disabled по техническим причинам, связанным с anti-bot/недоступностью стабильной серверной проверки.
- Успешная регистрация автоматизируется до отправки формы; сценарий завершения регистрации помечен @Disabled с явным объяснением причины.
- Negative-сценарий поиска остаётся активным и помечается @KnownDefect, а не обычным defect-тегом.
- KnownDefect не маскирует WebDriver/Timeout/locator errors.
- При изменении production-поведения сначала определяется: дефект теста, дефект продукта, инфраструктурный сбой или новая ожидаемая функциональность.

14. Авторизационные negative API сценарии

API	Сценарий	Ожидаемый результат
Token	Получение guest token без Authorization	HTTP 401
Browse / Product	Запрос валидной категории без Bearer token	HTTP 401
Inventory	Запрос валидного productId без Bearer token	HTTP 401

15. Ключевые UI negative сценарии регистрации

Сценарий	Ожидаемое поведение
Invalid / empty email	Validation error; Create Account disabled
Empty First Name + Last Name	Соответствующие validation errors; button disabled
Empty Password	Password validation error; button disabled
Empty Zip Code	Zip Code validation error; button disabled
Empty Birthday + Terms accepted	Create Account disabled

Сценарий	Ожидаемое поведение
Birthday selected + Terms not accepted	Create Account disabled

16. Риски и меры снижения

Риск	Влияние	Мера
Изменение production UI/API	Падение локаторов или контрактов	Page/Controller/DTO изолированы; анализ по Allure и логам.
Anti-bot protection	Блокировка UI-сценариев	@Disabled для технически недоступных сценариев; явное документирование.
Known defect продукта	Ожидаемое assertion failure	@KnownDefect strict-модель; unexpected pass требует перепроверки.
Динамические товарные данные	Невалидный SKU / изменённая доступность	Динамический выбор productId/SKU через API.
Eventual consistency	Состояние API обновляется не сразу	Awaitility с bounded polling.
Сетевые / production сбои	Ложные UI failures	Раздельные API/UI suite, повторный анализ, artifacts и stack traces.
Секреты и credentials	Риск утечки	Локально: ~/.gradle/gradle.properties вне репозитория; CI: GitHub Secrets; секреты отсутствуют в коде и Git history.

17. Финальная проверка текущей версии

После последнего overlay и Gradle refactoring подтверждены:

- Полный API suite - passed без ручной передачи -Dguest.header.auth.
- Полный UI suite - passed после стабилизации location / cookie / email overlays.
- Selenium WebDriver 4.50.0; targeted Search и Men's catalog scenarios - passed.
- Custom Gradle Test tasks выполняются без Test.classpath / Test.testClassesDirs deprecation warnings.
- Guest Basic credential локально читается из ~/.gradle/gradle.properties вне репозитория.
- KnownDefectExtension проверен; старый @Tag("defect") и defect Gradle task удалены.
- Authorization-negative API scenarios возвращают HTTP 401; Terms/Birthday UI scenarios - passed.

17.1 Актуальная инфраструктурная конфигурация

Gradle tasks allTests / allTestsRemote заменили allExceptDefect / allExceptDefectRemote. Глобальный parallel UI run не используется; homePageTestsParallel остаётся отдельным экспериментальным task.

18. Критерии успешного завершения

- Проект успешно компилируется (`testClasses`).
- API test suite проходит без функциональных ошибок.
- UI test suite проходит без функциональных ошибок с учётом @Disabled и @KnownDefect semantics.
- Критический API E2E lifecycle завершается успешно.
- API и UI формируют Allure results.
- Единый Allure Report создаётся и публикуется через GitHub Pages.
- В CI отсутствуют необъяснённые функциональные failures.
- Credentials не хранятся в репозитории.

19. Поддержка и актуализация плана

- Test Plan обновляется при существенном изменении архитектуры тестов, CI/CD, набора API/UI сценариев или внешних ограничений сайта.
- Конкретная версия браузера не фиксируется как обязательное ограничение; используется актуальная поддерживаемая версия.
- При добавлении новых test suites синхронно обновляются README и данный Test Plan.
- После изменения количества тестов итоговые числа проверяются по Gradle XML test results, а не рассчитываются вручную.

20. Ответственный

Илья Сидорычев
GitHub: https://github.com/ISmeneger
Repository: https://github.com/ISmeneger/American_Eagle_java_automation